Quiz — 2.2.1 Programming Techniques — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

Section A (1 mark each = 8)

Q	Ans	Why
A1	**C**	`do...until` is post-conditioned — the condition is tested at the **end**, so the body always runs at least once. `while` may run zero times.
A2	**B**	The defining difference: a **function returns a single value** (usable in an expression); a procedure returns none.
A3	**C**	With no reachable base case, calls never stop; each pushes a stack frame until memory is exhausted → **stack overflow**.
A4	**B**	By reference passes the **memory address** of the original, so changes affect the original.
A5	**B**	A variable declared inside a subroutine is **local** — visible only within it.
A6	**C**	**Encapsulation** = bundling data + methods with private attributes accessed via public methods (data hiding).
A7	**B**	Same name **and** parameters replacing an inherited method = **overriding** (overloading = same name, different parameters).
A8	**C**	A **breakpoint** pauses execution at a chosen line so variables can be inspected.

Section B (12)

B1. [2] - **Base case** — the condition that stops the recursion, with no further recursive call (1). - **General / recursive case** — calls the subroutine itself, moving closer to the base case (1).

B2. [3] - Output line 1: **5** (1) — inside the procedure the local `total` holds 5. - Output line 2: **10** (1) — outside, the global `total` is unchanged. - Explanation (1): the assignment inside `update()` creates/uses a **local** variable `total` whose scope is the procedure only; it **shadows** the global but does not alter it, so after the call the global is still 10.

B3. [3] - After `addOneByVal(x)`: **x = 3** (1) because a **copy** of the value is passed; `n` is incremented but the original `x` is **unchanged** (1 for either by-value point). - After `addOneByRef(x)`: **x = 4** (1) because the **memory address** of `x` is passed, so updating `n` updates the original `x`.

(Mark points: by value → original unchanged; by reference → original can be / is changed. The *consequence* is the mark, not just "a copy is made".)

B4. [2] Trace of `factorial(4)` (frames pushed then popped, LIFO):

```
factorial(4) = 4 * factorial(3)
factorial(3) = 3 * factorial(2)
factorial(2) = 2 * factorial(1)
factorial(1) = 1            (base case, n <= 1)
```

Unwinding: *21 = 2 → 32 = 6 → 46 = 24. - 1 mark for correct working/returns at each level; 1 mark for final answer 24**.

B5. [1] Any one: fewer side effects / no accidental change elsewhere; memory freed when the subroutine exits; more modular / reusable / easier to debug and maintain.

B6. [1] A **class** is a template/blueprint defining the attributes and methods; an **object** is a specific **instance** of that class created at run time. (Both halves needed for the mark.)

Section C (5)

Model answer:

```
class BankAccount
    private balance

    public procedure new()
        balance = 0                 // (a)
    endprocedure

    public procedure deposit(amount) // (b)
        balance = balance + amount
    endprocedure

    public function getBalance()      // (c)
        return balance
    endfunction
endclass
```

Mark points (1 each, max 5): 1. Constructor sets `balance = 0`. 2. `deposit` declared with a parameter `amount` (procedure). 3. `deposit` body adds `amount` to `balance` (`balance = balance + amount`). 4. `getBalance` declared as a **function** (it returns a value). 5. `getBalance` body uses `return balance`.

(Accept correct OCR-style syntax variants, e.g. `public function` / `public procedure`. Using a function for `deposit` or a procedure for `getBalance` loses the relevant point since a value must be returned by `getBalance`.)

Total: 8 + 12 + 5 = 25.